

Security in Software Applications

Homework 3

Rando, 1985084

A.A. 2024/2025

1 Echidna

Echidna is a specialized fuzzer designed for identifying potential security vulnerabilities in Ethereum smart contracts by rigorously testing Solidity code against developer-defined properties. Its operational methodology involves generating a large number of diverse, randomized inputs and subsequently verifying whether the defined invariants remain consistent under various conditions. In this context, invariants represent critical system properties that are expected to hold true given specific preconditions. Echidna evaluates these Solidity properties by executing the smart contract within a sandboxed environment. Should an invariant violation occur, the tool reports concrete counterexamples that precisely illustrate the failure, thereby aiding in the robust security assessment of smart contracts.

2 Solution (Part 1 and 2 of the Assignment)

2.1 Contract Invariants

The `Person.sol` contract, which is the subject of verification, includes a constructor. Due to Echidna's current limitation in directly assigning arguments to a contract's constructor, it was necessary to develop a `TestContract`. This `TestContract` is specifically designed to initialize an instance of the `Person` contract with the required constructor parameters, thereby enabling its subsequent fuzzer-based verification by Echidna.

2.1.1 Marriage Relationship

To ensure the integrity of the mutual marriage relationship, our initial step involves defining a corresponding invariant. In Echidna's notation, this can be expressed as follows:

```
function echidna_symmetricMarriage() public view returns (bool) {
    if (!isMarried || spouse == address(0)) return true;
    return Person(spouse).getSpouse() == address(this);
}
function echidna_spouseAddressConsistency() public view returns (bool) {
    return isMarried? spouse != address(0) : spouse == address(0);
}
```

```

}
function echidna_notSelfMarried() public view returns (bool) {
    return spouse != address(this);
}

```

The initial invariant is specifically designed to enforce the primary property concerning the mutual marriage relationship. The two subsequent invariants, however, are utilized to ensure consistency between the `isMarried` and `spouse` fields within the `Person` contract. It is important to note that the `getSpouse` function has been modified to a `view` function, allowing it to be called without altering the contract's state, a common requirement for functions invoked within invariants.

2.1.2 Subsidy

To ensure the accurate calculation and verification of the `state_subsidy` allocated to each individual, the following constants were initially defined:

```

uint constant ELDERLY_SUBSIDY = 600;
uint constant RETIREMENT_AGE = 65;

```

Subsequently, the `haveBirthday` function required modification to ensure the state subsidy is accurately adjusted in accordance with the updated age:

```

function haveBirthday() public {
    age++;
    updateSubsidy();
}
function updateSubsidy() public {
    if (isMarried)
        state_subsidy = DEFAULT_SUBSIDY * 70/100;
    else
        state_subsidy = age < RETIREMENT_AGE ?
            DEFAULT_SUBSIDY : ELDERLY_SUBSIDY;
}

```

Finally:

```

function echidna_rightSubsidy() public view returns (bool) {
    if (isMarried) return state_subsidy == DEFAULT_SUBSIDY * 70 / 100;
    return state_subsidy == (age < RETIREMENT_AGE ?
        DEFAULT_SUBSIDY : ELDERLY_SUBSIDY);
}
function echidna_validAge() public view returns (bool) {
    return age >= 0;
}

```

The first function ensures the requested property, and the other is extra.

2.2 Fixes

The contract lacked rigorous consistency checks. The first correction I made in the code was to add the following preconditions:

- **marry** (method to marry a Person given its address):
 - `require(new_spouse != address(0));`
 - `require(new_spouse != address(this));`
 - `require(new_spouse != spouse).`
- **divorce** (method to divorce from the current spouse):
 - `require(isMarried);`
 - `require(spouse != address(0));`
- **setSpouse** (method to change the address of the spouse):
 - `require(sp != address(this));`
 - `require(isMarried ? sp != address(0) : sp == address(0)).`

The other addition does not reduce to simple preconditions. Codes were added to ensure that every time the **marry** and **divorce** methods were used, they were also used on the other partner (if needed, of course, to avoid recursion).

2.3 Echidna Results

The execution of the Echidna on the code before and after the fixes are shown in Figure 1 (a)-(b) and Figure 2 respectively. Further details can be found in the dedicated GitHub page.

```

[ Echidna 2.2.6 ]
Workers: 0/4 Unique instructions: 589 Chain ID: -
Seed: 2080391222668624937 Unique codehashes: 1 Fetched contracts: 0/6
Calls/s: 80 Corpus size: 2 seqs Fetched slots: 0/0
Gas/s: 84331 New coverage: 4s ago
Total calls: 404/50000 Slither succeeded

Tests (3)
echidna_spouseAddressConsistency: FAILED! with ReturnFalse
Call sequence:
1. TestPerson.marry(0x0)

echidna_symmetricMarriage: FAILED! with ErrorRevert
Call sequence:
1. TestPerson.marry(0x0)

Traces:
call 0x0000000000000000000000000000000000000000000000000000000000000000::getSpouse() (/working.sol:68)
  Lc (0x)

Log (9)
[2025-04-01 12:09:12.37] [Worker 0] Test limit reached. Stopping.
[2025-04-01 12:09:09.26] [Worker 3] Test limit reached. Stopping.
[2025-04-01 12:09:07.97] [Worker 1] Test limit reached. Stopping.
[2025-04-01 12:09:07.96] [Worker 2] Test limit reached. Stopping.
[2025-04-01 12:09:07.96] [Worker 2] New coverage: 589 instr, 1 contracts, 2 seqs in corpus
[2025-04-01 12:09:07.92] [Worker 0] New coverage: 589 instr, 1 contracts, 1 seqs in corpus
[2025-04-01 12:09:07.90] [Worker 3] Test echidna_spouseAddressConsistency falsified!
[2025-04-01 12:09:07.85] [Worker 0] Test echidna_notSelfMarried falsified!
[2025-04-01 12:09:07.83] [Worker 0] Test echidna_symmetricMarriage falsified!

Campaign complete, C-c or esc to exit

```

(a) Echidna detecting invariant violations

```

echidna_symmetricMarriage: FAILED! with ErrorRevert
Call sequence:
1. TestPerson.marry(0x0)

Traces:
call 0x0000000000000000000000000000000000000000000000000000000000000000::getSpouse() (/working.sol:68)
  Lc (0x)

echidna_notSelfMarried: FAILED! with ReturnFalse
Call sequence:
1. TestPerson.marry(0xa329c0648769a73afac7f9381e08fb43dbea72)

Log (9)

```

(b) Echidna detecting invariant violations

Figure 1: Echidna analysis.

```

[ Echidna 2.2.6 ]
Workers: 0/4 Unique instructions: 1112 Chain ID: -
Seed: 4367064663245067340 Unique codehashes: 1 Fetched contracts: 0/6
Calls/s: 7169 Corpus size: 5 seqs Fetched slots: 0/0
Gas/s: 2532321 New coverage: 5s ago
Total calls: 50186/50000 Slither succeeded

Tests (3)
echidna_notSelfMarried: passing
echidna_symmetricMarriage: passing
echidna_spouseAddressConsistency: passing

Log (9)
[2025-04-01 13:37:58.02] [Worker 2] Test limit reached. Stopping.
[2025-04-01 13:37:57.86] [Worker 0] Test limit reached. Stopping.
[2025-04-01 13:37:57.76] [Worker 1] Test limit reached. Stopping.
[2025-04-01 13:37:57.72] [Worker 3] Test limit reached. Stopping.
[2025-04-01 13:37:52.47] [Worker 2] New coverage: 1112 instr, 1 contracts, 5 seqs in corpus
[2025-04-01 13:37:51.66] [Worker 1] New coverage: 1045 instr, 1 contracts, 4 seqs in corpus

Campaign complete, C-c or esc to exit

```

Figure 2: Echidna confirms all properties hold after fixing